Azure_lab02:

Application Gateway Traffic Light Left! Right!

1. The lab and the method

Goal. Deploy an Azure Application Gateway in front of two backend VMs and prove it
distributes traffic across both — one browser hit returns `Hello World from host myVM1!`,
the next returns `Hello World from host myVM2!`.

Source. Microsoft quickstart
Direct web traffic with Azure Application Gateway — Azure CLI.

Method: Azure CLI from Azure Cloud Shell (Bash).

What gets built.

Resource	Name	Notes
Resource group	`myResourceGroupAG`	
Virtual network	`myVNet`	`10.21.0.0/16`
Gateway subnet	`myAGSubnet`	`10.21.0.0/24` — may hold only application gateways
Backend subnet	`myBackendSubnet`	`10.21.1.0/24`
Public IP	`myAGPublicIPAddress`	Standard SKU, static (required by v2)
NSG	`myNSG`	Attached to both NICs
NAT gateway	`myNatGateway`	Outbound internet for the backend subnet
VMs	`myVM1`, `myVM2`	Ubuntu 22.04, `Standard_D2s_v6`, no public IP
Application Gateway	`myAppGateway`	`Standard_v2`, capacity 2, listener on :80

Traffic path: internet → gateway public IP :80 → listener → routing rule → backend pool →
nginx :80 on each VM → Node.js/Express :3000 → hostname response.

---

2. Commands

Run in Azure Cloud Shell (Bash), in order.

Variables and resource group

```
LOC=germanywestcentral
RG=myResourceGroupAG

az group create --name $RG --location $LOC
```

Network

```
az network vnet create -g $RG -n myVNet -l $LOC \
  --address-prefix 10.21.0.0/16 \
  --subnet-name myAGSubnet --subnet-prefix 10.21.0.0/24

az network vnet subnet create -g $RG --vnet-name myVNet \
  -n myBackendSubnet --address-prefix 10.21.1.0/24

az network public-ip create -g $RG -n myAGPublicIPAddress \
  --allocation-method Static --sku Standard -l $LOC
```

NSG with the port 80 rule

```
az network nsg create -g $RG -n myNSG -l $LOC

az network nsg rule create -g $RG --nsg-name myNSG -n AllowHT
TPFromAppGw \
  --priority 100 --direction Inbound --access Allow --protoco
l Tcp \
```

```
  --source-address-prefixes 10.21.0.0/24 --source-port-ranges
'*' \
  --destination-address-prefixes '*' --destination-port-range
s 80
```

> Source is the gateway subnet only — the VMs never need traffic from anywhere else.

NAT gateway for outbound access

```
az network public-ip create -g $RG -n myNatPip -l $LOC \
  --sku Standard --allocation-method Static

az network nat gateway create -g $RG -n myNatGateway -l $LOC
\
  --public-ip-addresses myNatPip --idle-timeout 10

az network vnet subnet update -g $RG --vnet-name myVNet \
  -n myBackendSubnet --nat-gateway myNatGateway
```

> Create this **before** the VMs. Azure has retired default outbound access, so a VM with no
> public IP has no route to the internet and `apt` cannot install anything. Outbound only —
> the VMs stay unreachable from outside.

cloud-init file

```
cat > cloud-init.txt <<'EOF'
#cloud-config
package_upgrade: true
packages:
  - nginx
  - nodejs
  - npm
```

```
write_files:
  - owner: www-data:www-data
    path: /etc/nginx/sites-available/default
    content: |
      server {
        listen 80;
        location / {
          proxy_pass http://localhost:3000;
          proxy_http_version 1.1;
          proxy_set_header Upgrade $http_upgrade;
          proxy_set_header Connection keep-alive;
          proxy_set_header Host $host;
          proxy_cache_bypass $http_upgrade;
        }
      }
  - owner: azureuser:azureuser
    path: /home/azureuser/myapp/index.js
    content: |
      var express = require('express')
      var app = express()
      var os = require('os');
      app.get('/', function (req, res) {
        res.send('Hello World from host ' + os.hostname() +
'!')
      })
      app.listen(3000, function () {
        console.log('Hello world app listening on port 300
0!')
      })
runcmd:
  - service nginx restart
  - cd "/home/azureuser/myapp"
  - npm init -y
  - npm install express@4
  - nohup nodejs index.js &
EOF
```

```
head -1 cloud-init.txt   # must print: #cloud-config
```

> Quoting `'EOF'` stops Bash expanding `$http_upgrade` and `$host` before the file is written.

NICs and VMs

```
az vm list-skus -l $LOC --size Standard_D2s_v6 -o table   # e
mpty? use Standard_D2s_v5

for i in 1 2; do
  az network nic create -g $RG -n myNic$i --vnet-name myVNet
\
    --subnet myBackendSubnet --network-security-group myNSG
  az vm create -g $RG -n myVM$i -l $LOC \
    --nics myNic$i --image Ubuntu2204 --size Standard_D2s_v6
\
    --public-ip-address "" --admin-username azureuser \
    --generate-ssh-keys --custom-data "$PWD/cloud-init.txt"
done
```

> Pass `--custom-data` as an absolute path. Given a path it cannot resolve, the CLI silently
> uploads the literal string instead, and cloud-init ignores it.

Application Gateway

```
address1=$(az network nic show -n myNic1 -g $RG --query "ipCo
nfigurations[0].privateIPAddress" -o tsv)
address2=$(az network nic show -n myNic2 -g $RG --query "ipCo
nfigurations[0].privateIPAddress" -o tsv)
echo "$address1 $address2"

az network application-gateway create -g $RG -n myAppGateway
```

```
-l $LOC \
  --capacity 2 --sku Standard_v2 \
  --public-ip-address myAGPublicIPAddress \
  --vnet-name myVNet --subnet myAGSubnet \
  --servers "$address1" "$address2" --priority 100
```

> Takes 5–30 minutes.

Verify

```
az network application-gateway show-backend-health -g $RG -n
myAppGateway \
  --query "backendAddressPools[].backendHttpSettingsCollectio
n[].servers[].{server:address,health:health,why:healthProbeLo
g}" \
  -o table
```

> Plain `-o table` prints nothing here — the response is nested too deeply for the formatter.
> Both servers must read `Healthy` before the browser will work.

```
IP=$(az network public-ip show -g $RG -n myAGPublicIPAddress
--query ipAddress -o tsv)
for i in $(seq 1 6); do curl -s http://$IP; echo; done
```

Expected: `Hello World from host myVM1!` and `Hello World from host myVM2!`, after a refresh, alternating.

Cleanup

```
az group delete --name $RG --yes --no-wait
```

---

3. Problems encountered

502 Bad Gateway, both backends Unhealthy. The probe log said *"Cannot connect to backend server."* Root cause: nginx, Node and npm were never installed — `npm: not found` and `Unit nginx.service not found` on both VMs, and cloud-init had finished in 14 seconds, far too fast to have installed anything. With no default outbound access, `apt` could not reach the Ubuntu archives. Fixed by attaching a NAT gateway to the backend subnet. Not in Microsoft's guide, which predates the retirement of default outbound access.

NSG missing a port 80 rule. The quickstart creates an NSG but opens nothing on it, so gateway health probes are dropped. Added an explicit inbound allow from `10.21.0.0/24`.

Express 5.x against Node 12. Ubuntu 22.04's `nodejs` package is v12; a bare `npm install express` now resolves to 5.x, which requires Node 18+. The app fails to start, nginx has nothing on :3000 to proxy to, and returns 502. Pinned to `express@4`.

Empty output from `show-backend-health -o table`. Looked like a failure, was a formatting limitation — the nested response needs an explicit `--query`. Cost time chasing a non-problem.

Debugging with no SSH. The VMs have no public IP and the NSG blocks port 22, so `az vm run-command invoke` was the only way in:

```
az vm run-command invoke -g $RG -n myVM1 --command-id RunShel
lScript \
  --query "value[0].message" -o tsv \
  --scripts "cloud-init status; systemctl is-active nginx; pg
rep -a node; curl -s -m 5 localhost"
```

Wrap it in `--query "value[0].message" -o tsv`, otherwise the output is buried in a JSON envelope.

Also worth noting: `npm init` without `-y` blocks waiting on stdin inside cloud-init, and
the `write_files` block in Microsoft's YAML splits `owner` and `path` into separate list items, which is malformed. Both corrected above.